

Universidade Federal de Goiás

Instituto de Informática — Pós-Graduação em Sistemas e Agentes Inteligentes

Disciplina: Construção de APIs para Inteligência Artificial

API de Inteligência Artificial Acadêmica

Documentação Técnica Detalhada dos Arquivos do Projeto

Trabalho Final — 2026

Aluna: Eliane Maria de Abreu Apolinario

Professor: Rogério Rodrigues Carvalho

Repositório público no GitHub

Link para o vídeo no canal do Youtube

1. Visão Geral do Projeto

A API de Inteligência Artificial Acadêmica é uma aplicação desenvolvida com FastAPI e integrada ao Hermes Agent (NousResearch), que disponibiliza dois serviços de IA para suporte a estudantes de pós-graduação. O projeto aplica na prática os conceitos abordados na disciplina: validação de dados, tratamento de erros, logs, segurança e versionamento.

1.1 Objetivo

Desenvolver uma API REST funcional com pelo menos dois endpoints de Inteligência Artificial distintos, demonstrando boas práticas de desenvolvimento de APIs. Os dois serviços implementados são:

- **POST /v1/ask** — Agente de IA que responde dúvidas sobre o conteúdo das aulas consultando PDFs na pasta data/aulas/.
- **POST /v1/tasks** — Agente de IA que consulta tarefas, documentos e prazos de entrega a partir de arquivo em data/tarefas/.

1.2 Tecnologias Utilizadas

Tecnologia	Função
FastAPI	Framework web para construção da API REST
Hermes Agent (NousResearch)	Motor de agentes de IA com acesso a ferramentas de leitura de arquivos
Pydantic v2	Validação automática de dados de entrada e saída
Uvicorn	Servidor ASGI para execução da aplicação em produção/desenvolvimento
Python logging	Sistema de logs em terminal e arquivo persistente
pytest + httpx	Testes automatizados dos endpoints

1.3 Estrutura de Pastas

```
ai_api_project/
├── app/
│   ├── agents/
│   │   ├── content_agent.py    ← Agente de conteúdo (PDFs de aula)
│   │   └── tasks_agent.py      ← Agente de tarefas e prazos
│   ├── api/
│   │   └── v1/
│   │       ├── content.py      ← Endpoint POST /v1/ask
│   │       └── tasks.py        ← Endpoint POST /v1/tasks
│   ├── core/
│   │   ├── config.py           ← Variáveis de ambiente e caminhos
│   │   ├── logger.py           ← Logger centralizado
│   │   ├── security.py         ← Autenticação via X-API-Key
│   │   └── main.py             ← Inicialização do FastAPI
│   └── data/
│       ├── aulas/              ← PDFs de conteúdo das aulas
│       └── tarefas/            ← Documento de tarefas (DOCX/PDF)
```

```
├── tests/
│   └── test_endpoints.py      ← 10 testes automatizados
├── logs/                     ← Gerado automaticamente
├── .env.example
├── requirements.txt
└── README.md
```

2. Arquivos de Configuração e Ambiente

.env.example

Caminho: `.env.example`

Descrição: Modelo do arquivo de variáveis de ambiente. Não deve ser commitado com valores reais. O desenvolvedor que clonar o repositório deve copiá-lo para `.env` e preencher com suas credenciais.

Responsabilidades:

- Define as quatro variáveis de ambiente necessárias para execução da API
- Instrui o desenvolvedor sobre quais chaves precisam ser obtidas (OpenRouter, chave própria da API)
- Documenta os valores padrão e opções disponíveis (níveis de log, modelos de IA)
- Protegido pelo `.gitignore` — a versão real (`.env`) nunca é enviada ao repositório

Código:

```
OPENROUTER_API_KEY=sua_chave_aqui
API_SECRET_KEY=sua_chave_secreta_aqui
AI_MODEL=deepseek/deepseek-v4-flash
LOG_LEVEL=INFO
```

requirements.txt

Caminho: `requirements.txt`

Descrição: Lista todas as dependências Python necessárias para execução do projeto. Permite que qualquer desenvolvedor instale o ambiente completo com um único comando: `pip install -r requirements.txt`.

Responsabilidades:

- Garante reprodutibilidade do ambiente em qualquer máquina
- Define versões mínimas de cada pacote para evitar incompatibilidades
- Inclui `hermes-agent` instalado diretamente do repositório GitHub da NousResearch

Código:

```
fastapi[standard]>=0.111.0
uvicorn[standard]>=0.30.0
python-dotenv>=1.0.0
pydantic>=2.7.0
hermes-agent @ git+https://github.com/NousResearch/hermes-agent.git
python-multipart>=0.0.9
```

.gitignore

Caminho: .gitignore

Descrição: Instrui o Git a ignorar arquivos que não devem ser versionados: credenciais, cache Python, logs e ambiente virtual.

Responsabilidades:

- Protege o arquivo .env com chaves reais de ser acidentalmente publicado no GitHub
- Evita versionamento de arquivos temporários do Python (__pycache__, .pyc)
- Impede que a pasta de logs e o ambiente virtual (.venv) sejam incluídos no repositório

Código:

```
.env
__pycache__/*
*.pyc
.pytest_cache/
logs/
.venv/
```

3. Camada Core (app/core/)

A camada core contém os módulos transversais da aplicação — componentes reutilizados por todas as outras camadas: configuração, logs e segurança. Esta separação segue a arquitetura em camadas apresentada na disciplina, garantindo manutenibilidade e desacoplamento.

config.py

Caminho: app/core/config.py

Descrição: Centraliza toda a configuração da aplicação lendo variáveis do arquivo .env via python-dotenv. É o único arquivo que acessa diretamente as variáveis de ambiente — todos os outros módulos importam as constantes daqui.

Responsabilidades:

- Carrega o arquivo .env automaticamente ao ser importado (load_dotenv())
- Expõe API_SECRET_KEY usada pela camada de segurança para autenticação
- Expõe AI_MODEL para que os agentes saibam qual modelo de IA usar
- Expõe LOG_LEVEL para configurar a verbosidade do sistema de logs
- Calcula os caminhos absolutos de DATA_DIR, AULAS_DIR e TAREFAS_DIR a partir da posição do próprio arquivo, garantindo funcionamento independente de onde a API é executada

Código:

```
import os
from dotenv import load_dotenv

load_dotenv()

API_SECRET_KEY = os.getenv('API_SECRET_KEY', '')
AI_MODEL        = os.getenv('AI_MODEL', 'anthropic/claude-sonnet-4-6')
LOG_LEVEL       = os.getenv('LOG_LEVEL', 'INFO')

DATA_DIR        = os.path.join(os.path.dirname(__file__), '..', '..', 'data')
AULAS_DIR       = os.path.join(DATA_DIR, 'aulas')
TAREFAS_DIR     = os.path.join(DATA_DIR, 'tarefas')
```

logger.py

Caminho: app/core/logger.py

Descrição: Configura o sistema de logs da aplicação com dois destinos simultâneos: saída no terminal (para desenvolvimento) e arquivo persistente logs/app.log (para monitoramento em produção). Todo módulo da aplicação obtém seu logger chamando get_logger(__name__).

Responsabilidades:

- Cria a pasta logs/ automaticamente se não existir (os.makedirs com exist_ok=True)
- Configura o nível de log dinamicamente a partir da variável LOG_LEVEL do .env
- Define formato padronizado: data/hora | nível | módulo | mensagem
- Registra simultaneamente no terminal (StreamHandler) e em arquivo (FileHandler)
- Usa encoding UTF-8 no arquivo para suportar caracteres especiais do português
- Expõe a função get_logger(name) para criação de loggers nomeados por módulo

Código:

```
import logging, os
from app.core.config import LOG_LEVEL

os.makedirs('logs', exist_ok=True)

logging.basicConfig(
```

```
level=getattr(logging, LOG_LEVEL, logging.INFO),
format='%(asctime)s | %(levelname)s | %(name)s | %(message)s',
handlers=[
    logging.StreamHandler(),
    logging.FileHandler('logs/app.log', encoding='utf-8'),
],
)

def get_logger(name: str) -> logging.Logger:
    return logging.getLogger(name)
```

Formato de log gerado: 2026-06-26 14:32:01 | INFO | app.agents.content_agent | Consultando agente de conteúdo | pergunta: O que é REST?...

security.py

Caminho: app/core/security.py

Descrição: Implementa a camada de segurança da API através de autenticação por chave de API (API Key). Qualquer endpoint que declare a dependência `require_api_key` exige que o cliente envie o header `X-API-Key` com a chave correta. Sem autenticação válida, a requisição é rejeitada com HTTP 401 antes de chegar à lógica de negócio.

Responsabilidades:

- Declara o esquema de segurança `APIKeyHeader` que o FastAPI usa para gerar automaticamente o campo no Swagger
- A função `require_api_key` é uma dependência injetada nos endpoints via `Depends()`
- Verifica primeiro se a `API_SECRET_KEY` está configurada no servidor — se não estiver, retorna HTTP 500
- Compara a chave recebida no header com a chave configurada — se diferente, retorna HTTP 401
- O header `WWW-Authenticate` informa ao cliente o esquema de autenticação esperado
- Endpoints que não declaram esta dependência são públicos (como o health check GET /)

Código:

```
from fastapi import Security, HTTPException, status
from fastapi.security.api_key import APIKeyHeader
from app.core.config import API_SECRET_KEY

api_key_header = APIKeyHeader(name='X-API-Key', auto_error=False)

async def require_api_key(api_key: str = Security(api_key_header)):
    if not API_SECRET_KEY:
        raise HTTPException(status_code=500,
                             detail='API_SECRET_KEY não configurada no servidor.')
    if api_key != API_SECRET_KEY:
        raise HTTPException(status_code=401,
                             detail='Chave de API inválida ou ausente.',
```

```
headers={'WWW-Authenticate': 'ApiKey'})  
return api_key
```

4. Camada de Agentes (app/agents/)

Os agentes são o núcleo de inteligência artificial do projeto. Cada agente encapsula um AIAgent do Hermes Agent configurado com um system prompt especializado para uma função específica. O Hermes Agent opera em loop autônomo: lê arquivos, raciocina sobre o conteúdo e formula a resposta sem necessitar de RAG ou banco vetorial.

Por que não é necessário RAG neste projeto?

O Hermes Agent possui ferramentas nativas de leitura de arquivos do sistema operacional. A cada consulta, o agente recebe a lista de arquivos disponíveis no system prompt e usa sua ferramenta de leitura para acessar os PDFs diretamente. Isso é suficiente para o escopo acadêmico proposto — com poucos documentos, a leitura direta é mais simples e igualmente eficaz que um pipeline de embeddings e banco vetorial.

content_agent.py

Caminho: app/agents/content_agent.py

Descrição: Implementa o agente de conteúdo acadêmico. Este agente responde dúvidas dos alunos consultando os PDFs disponíveis na pasta data/aulas/. A cada chamada, uma nova instância de AIAgent é criada para garantir isolamento entre requisições (thread-safety).

Responsabilidades:

- Define SYSTEM_PROMPT especializado: instruções que moldam o comportamento do agente como assistente acadêmico didático e honesto
- list_available_pdfs(): varre data/aulas/ e retorna lista de arquivos .pdf — usado tanto pelo agente quanto pelo endpoint para informar as fontes disponíveis
- ask_content_agent(): função principal que monta o prompt enriquecido (system prompt + lista de PDFs) e instancia o AIAgent
- O prompt enriquecido instrui o agente a usar a ferramenta de leitura de arquivos antes de responder
- Parâmetros do AIAgent: quiet_mode=True (sem output no terminal), skip_memory=True (sem persistência entre sessões), max_iterations=15 (limite de iterações do loop de raciocínio)
- Lança FileNotFoundError se a pasta data/aulas/ estiver vazia — tratado pelo endpoint como HTTP 404
- Registra logs antes e após a execução para rastreabilidade

Código:

```
SYSTEM_PROMPT = '''Você é um assistente acadêmico especializado no conteúdo das aulas.
Responda dúvidas com base nos materiais de estudo disponíveis na pasta de aulas.
Seja claro, didático e objetivo. Se a dúvida não estiver coberta pelo material,
diga isso honestamente em vez de inventar uma resposta.'''

def ask_content_agent(question: str) -> str:
    pdfs = list_available_pdfs()
    if not pdfs:
        raise FileNotFoundError('Nenhum material de aula encontrado.')
    enriched_prompt = f'{SYSTEM_PROMPT}\n\nMateriais: {pdfs}...'
    agent = AIAgent(
        model=AI_MODEL,
        ephemeral_system_prompt=enriched_prompt,
        quiet_mode=True, skip_memory=True, max_iterations=15
    )
    return agent.chat(question)
```

tasks_agent.py

Caminho: app/agents/tasks_agent.py

Descrição: Implementa o agente de gerenciamento de tarefas e prazos. Segue a mesma estrutura do content_agent.py, mas com system prompt especializado em consulta de prazos de entrega e documentos que precisam ser preenchidos. Aceita arquivos .docx, .pdf e .txt na pasta data/tarefas/.

Responsabilidades:

- Define SYSTEM_PROMPT orientado a precisão com datas, nomes de documentos e organização de prazos
- O agente está instruído a considerar a data atual ao avaliar tarefas próximas do prazo ou vencidas
- list_task_files(): aceita extensões .docx, .pdf e .txt — flexível para diferentes formatos do documento de tarefas
- ask_tasks_agent(): mesma estrutura do agente de conteúdo, mas aponta para TAREFAS_DIR
- Lança FileNotFoundError se data/tarefas/ estiver vazia
- Cria nova instância de AIAgent por chamada — sem compartilhamento de estado entre requisições

Código:

```
SYSTEM_PROMPT = '''Você é um assistente especializado em tarefas e prazos.
Consulte o documento disponível e responda sobre:
- Quais tarefas precisam ser entregues
- Documentos que devem ser preenchidos
- Datas limite de entrega
- Tarefas próximas do prazo ou vencidas (use a data de hoje)'''

def list_task_files() -> list[str]:
```



```
extensions = ('.docx', '.pdf', '.txt')
return [f for f in os.listdir(TAREFAS_DIR)
        if f.lower().endswith(extensions)]
```

5. Camada de Endpoints (app/api/v1/)

Os endpoints são a interface pública da API — onde as requisições HTTP chegam, são validadas, encaminhadas aos agentes e as respostas são formatadas. O prefixo /v1/ implementa o versionamento de rotas, garantindo compatibilidade retroativa se uma versão futura (v2) for criada.

content.py

Caminho: app/api/v1/content.py

Descrição: Define o endpoint POST /v1/ask para consulta ao agente de conteúdo das aulas. Toda a lógica de IA está no agente — este arquivo é responsável apenas pela interface HTTP: validação de entrada, autenticação, chamada ao agente e formatação da resposta.

Responsabilidades:

- ContentRequest: modelo Pydantic com campo question (min 5, max 1000 caracteres) — validação automática com HTTP 422 em caso de violação
- ContentResponse: modelo de resposta com answer (texto do agente) e sources_available (lista dos PDFs consultados)
- A rota declara Depends(require_api_key) — autenticação obrigatória antes de qualquer processamento
- Captura FileNotFoundError e retorna HTTP 404 com mensagem descritiva
- Captura Exception genérica e retorna HTTP 500 — evita exposição de erros internos ao cliente
- O decorator @router.post inclui summary e description usados na documentação Swagger automática
- Registra erros no log para diagnóstico sem expô-los na resposta HTTP

Código:

```
class ContentRequest(BaseModel):
    question: str = Field(..., min_length=5, max_length=1000,
                          description='Dúvida sobre o conteúdo das aulas.',
                          examples=['O que é uma API REST?'])

class ContentResponse(BaseModel):
    answer: str
    sources_available: list[str]

@router.post('/ask', response_model=ContentResponse)
async def ask_content(body: ContentRequest, _=Depends(require_api_key)):
    try:
```

```
answer = ask_content_agent(body.question)
return ContentResponse(answer=answer,
                        sources_available=list_available_pdfs())
except FileNotFoundError as exc:
    raise HTTPException(status_code=404, detail=str(exc))
except Exception as exc:
    raise HTTPException(status_code=500, detail='Erro interno.')
```

tasks.py

Caminho: app/api/v1/tasks.py

Descrição: Define o endpoint POST /v1/tasks para consulta ao agente de tarefas. Segue a mesma estrutura do content.py, com modelos de dados próprios e chamada ao tasks_agent. O campo files_consulted na resposta informa quais arquivos de tarefas foram disponibilizados ao agente.

Responsabilidades:

- TaskRequest: mesmo esquema de validação do ContentRequest (question, 5-1000 chars)
- TaskResponse: retorna answer e files_consulted (arquivos da pasta data/tarefas/)
- Mesmo tratamento de erros: FileNotFoundError → 404, Exception → 500
- Mesma dependência de autenticação: Depends(require_api_key)
- Os dois endpoints são intencionalmente distintos: diferentes agentes, diferentes system prompts, diferentes pastas de dados e diferentes campos de resposta

Código:

```
class TaskRequest(BaseModel):
    question: str = Field(..., min_length=5, max_length=1000,
                          description='Pergunta sobre tarefas, documentos ou prazos.',
                          examples=['Quais tarefas vencem essa semana?'])

class TaskResponse(BaseModel):
    answer: str
    files_consulted: list[str]

@router.post('/tasks', response_model=TaskResponse)
async def ask_tasks(body: TaskRequest, _=Depends(require_api_key)):
    try:
        answer = ask_tasks_agent(body.question)
        return TaskResponse(answer=answer,
                            files_consulted=list_task_files())
    except FileNotFoundError as exc:
        raise HTTPException(status_code=404, detail=str(exc))
    except Exception as exc:
        raise HTTPException(status_code=500, detail='Erro interno.')
```

6. Ponto de Entrada da Aplicação (app/main.py)

O main.py é o ponto de entrada da aplicação. É o arquivo que o uvicorn executa ao iniciar o servidor. Ele instancia o objeto FastAPI, registra os routers e define os handlers globais.

main.py

Caminho: app/main.py

Descrição: Inicializa a aplicação FastAPI com metadados completos (título, descrição, versão), registra os dois routers de endpoints com o prefixo /v1, define um handler global de exceções não tratadas e expõe o endpoint público GET / (health check).

Responsabilidades:

- Instancia FastAPI com title, description e version — usados na interface Swagger em /docs
- include_router(content.router, prefix='/v1', tags=['Conteúdo das Aulas']): registra as rotas do módulo content.py sob /v1 e agrupa no Swagger
- include_router(tasks.router, prefix='/v1', tags=['Tarefas e Prazos']): idem para tasks.py
- global_exception_handler: captura qualquer Exception não tratada pelos endpoints, registra no log e retorna HTTP 500 sem expor detalhes internos
- health_check (GET /): endpoint público sem autenticação que permite verificar se a API está no ar — responde {status: ok, version: 1.0.0}

Código:

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse
from app.api.v1 import content, tasks

app = FastAPI(
    title='API de Inteligência Artificial Acadêmica',
    description='Dois serviços de IA para suporte acadêmico.',
    version='1.0.0',
)

app.include_router(content.router, prefix='/v1',
                    tags=['Conteúdo das Aulas'])
app.include_router(tasks.router, prefix='/v1',
                    tags=['Tarefas e Prazos'])

@app.exception_handler(Exception)
async def global_exception_handler(request, exc):
    logger.exception('Exceção não tratada em %s', request.url.path)
    return JSONResponse(status_code=500,
                        content={'detail': 'Erro interno inesperado.'})

@app.get('/', tags=['Status'])
async def health_check():
    return {'status': 'ok', 'version': '1.0.0'}
```

7. Testes Automatizados (tests/test_endpoints.py)

O arquivo de testes garante que todos os cenários relevantes — dados válidos, dados inválidos, autenticação correta e incorreta, e situações de erro — funcionem conforme esperado. Os testes usam pytest com o TestClient do FastAPI, que simula requisições HTTP sem precisar de um servidor rodando.

Execução: `pytest tests/ -v`

Função de Teste	HTTP Esperado	O que verifica
<code>test_ask_sem_chave_retorna_401</code>	401	Requisição sem header X-API-Key
<code>test_tasks_sem_chave_retorna_401</code>	401	Idem para o endpoint /tasks
<code>test_ask_chave_invalida_retorna_401</code>	401	Header presente mas com valor errado
<code>test_ask_pergunta_muito_curta_retorna_422</code>	422	Pergunta com menos de 5 caracteres
<code>test_tasks_pergunta_muito_curta_retorna_422</code>	422	Idem para /tasks
<code>test_ask_sem_body_retorna_422</code>	422	Requisição sem corpo JSON
<code>test_ask_retorna_resposta_valida</code>	200	Resposta com answer e sources_available
<code>test_tasks_retorna_resposta_valida</code>	200	Resposta com answer e files_consulted
<code>test_ask_sem_pdfs_retorna_404</code>	404	Pasta data/aulas/ vazia
<code>test_tasks_sem_arquivo_retorna_404</code>	404	Pasta data/tarefas/ vazia
<code>test_health_check</code>	200	GET / retorna {status: ok}

Os testes funcionais (200 e 404) usam `unittest.mock.patch` para substituir as chamadas reais ao Hermes Agent por respostas controladas, tornando os testes rápidos, determinísticos e independentes de chaves de API ou arquivos de dados.

8. Requisitos do Trabalho Atendidos

Requisito	Implementação
2 endpoints de IA distintos	POST /v1/ask (agente de conteúdo) e POST /v1/tasks (agente de tarefas) — operações, agentes e dados completamente diferentes
Validação de dados	Pydantic v2 nos modelos ContentRequest e TaskRequest: campo question com min_length=5, max_length=1000. HTTP 422 automático em violações.
Tratamento de erros	FileNotFoundError → HTTP 404; Exception genérica → HTTP 500; autenticação inválida → HTTP 401; validação falha → HTTP 422
Logs	logger.py com dois handlers (terminal + arquivo logs/app.log), nível configurável via .env, formato com timestamp e módulo de origem
Segurança	security.py com autenticação via header X-API-Key usando APIKeyHeader do FastAPI. Todos os endpoints de IA são protegidos via Depends().
Versionamento	Prefixo /v1/ em todos os endpoints de IA, garantindo compatibilidade retroativa e permitindo criação de /v2/ no futuro
Documentação	Swagger automático em /docs gerado pelo FastAPI com modelos, exemplos e campo de autenticação integrado
Repositório GitHub	Código-fonte público com README completo contendo instruções de instalação, configuração e execução
Testes automatizados	11 testes em tests/test_endpoints.py cobrindo todos os cenários: 401, 404, 422, 200 e health check